PR1: Searching & Sorting (Student Array)

```
class Student {
    int rollNo;
    String name;
    float sgpa;

    public Student(int r, String n, float s) {
        rollNo = r;
        name = n;
        sgpa = s;
    }
}

class StudentDB {
    private Student[] students;
    private int size;

    public StudentDB(Student[] s) {
        students = s;
        size = s.length;
    }

    public void bubbleSortByRollNo() {
        int n = size;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (students[j].rollNo > students[j + 1].rollNo) {
                    Student temp = students[j];
                    students[j] = students[j + 1];
                    students[j + 1] = temp;
                }
            }
        }
    }

    public void display() {
        for (Student s : students) {
            System.out.println(s.rollNo + " " + s.name + " " + s.sgpa);
        }
    }
}
```

PR2: Linked List (Singly Linked List)

```
class SLLNode {
    int data;
    SLLNode next;

    public SLLNode(int data) {
        this.data = data;
        this.next = null;
    }
}

class SinglyLinkedList {
    SLLNode head;

    public void insertAtFront(int data) {
        SLLNode newNode = new SLLNode(data);
        newNode.next = head;
        head = newNode;
    }

    public void traverse() {
        SLLNode current = head;
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.next;
        }
        System.out.println();
    }
}
```

PR3: Stack ADT (Linked List Implementation)

```
class StackNode {
    int data;
    StackNode next;

    public StackNode(int data) {
        this.data = data;
    }
}

class StackADT {
    StackNode top;

    public void push(int data) {
        StackNode newNode = new StackNode(data);
        newNode.next = top;
        top = newNode;
    }

    public int pop() {
        if (top == null) {
            System.out.println("Stack Empty");
            return -1;
        }
        int element = top.data;
        top = top.next;
        return element;
    }
}
```

PR4: Expression Tree (Creation)

```
import java.util.Stack;

class BNode {
    String data;
    BNode left, right;

    public BNode(String data) {
        this.data = data;
    }
}

class ExpressionTree {
    public BNode create(String postfix) {
        Stack<BNode> stack = new Stack<>();
        for (int i = 0; i < postfix.length(); i++) {
            String token = String.valueOf(postfix.charAt(i));
            if (!isOperator(token)) {
                stack.push(new BNode(token));
            } else {
                BNode newNode = new BNode(token);
                newNode.right = stack.pop();
                newNode.left = stack.pop();
                stack.push(newNode);
            }
        }
        return stack.pop();
    }

    private boolean isOperator(String s) {
        return s.matches("[+\-*/^]");
    }
}
```

PR5: Binary Search Tree (Insertion)

```
class BstNode {
    int data;
    BstNode left, right;

    public BstNode(int data) {
        this.data = data;
    }
}

class BinarySearchTree {
    BstNode root;

    public void insert(int data) {
        root = insertRec(root, data);
    }

    private BstNode insertRec(BstNode current, int data) {
        if (current == null) {
            return new BstNode(data);
        }
        if (data < current.data) {
            current.left = insertRec(current.left, data);
        } else if (data > current.data) {
            current.right = insertRec(current.right, data);
        }
        return current;
    }
}
```

PR6/PR7: Graph Algorithms (Dijkstra's Shortest Path)

```
class Graph {
    int V;
    int[][] cost;

    public Graph(int vertices) {
        V = vertices;
        cost = new int[V][V];
    }

    public int minDistance(int[] dist, boolean[] visited) {
        int min = Integer.MAX_VALUE;
        int minIndex = -1;
        for (int v = 0; v < V; v++) {
            if (!visited[v] && dist[v] <= min) {
                min = dist[v];
                minIndex = v;
            }
        }
        return minIndex;
    }

    public void dijkstra(int src) {
        int[] dist = new int[V];
        boolean[] visited = new boolean[V];
        for (int i = 0; i < V; i++) dist[i] = Integer.MAX_VALUE;
        dist[src] = 0;
        for (int count = 0; count < V - 1; count++) {
            int u = minDistance(dist, visited);
            visited[u] = true;
            for (int v = 0; v < V; v++) {
                if (!visited[v] && cost[u][v] != 0 && dist[u] != Integer.MAX_VALUE && dist[u] + cost
                    dist[v] = dist[u] + cost[u][v];
                }
            }
        }
    }
}
```